Smart MCQ Solver — Semantic Similarity & Transformer-Based Answer Ranking

Author: Shivam | Student ID: 24f1002692

DL / GenAI Course Project · Term T22026

1. Executive Summary

The project builds an end-to-end pipeline that ranks the top-3 most probable answers for multiple-choice questions (MCQs), evaluated using the MAP@3 metric. The pipeline progresses through five milestones — classical NLP baselines (TF-IDF, Word2Vec), pretrained transformer embeddings and zero-shot classification, Retrieval-Augmented Generation (RAG) with a Wikipedia FAISS index, LoRA fine-tuning of BERT for multiple-choice classification, and sequence-classification ensembling of DeBERTa and RoBERTa. The final production model is a from-scratch Bidirectional GRU (BiGRU) with attention pooling, trained without any pretrained weights but augmented with RAG-retrieved Wikipedia context to compensate for the small training set and to reduce overfitting.

The dataset consists of 2,000 training questions across 5 options (A–E), with 200+ duplicate prompts identified and handled via a prompt-level (not row-level) train/validation split to prevent data leakage. The final RAG-augmented BiGRU model reached a validation MAP@3 of 0.996 after 25 epochs, while the confirmed public leaderboard score settled around 0.75, reflecting a distribution shift between the train/validation split and the hidden Kaggle test set.

Approach	Core Technique	Milestone	Notes
TF-IDF / Word2Vec	Sparse & dense lexical embeddings + cosine similarity	M1	Classical NLP baseline
BERT/RoBERTa + SBERT + Zero-shot	CLS pooling, sentence embeddings, BART-MNLI	M2	Pretrained, no fine-tuning
RAG (FAISS + Flan-T5)	Wikipedia passage retrieval + generative reader	M3	Adds external knowledge
LoRA Fine-tuned BERT	Parameter-efficient multiple-choice fine-tuning	M4	Task-adapted pretrained model
DeBERTa + RoBERTa Ensemble	Weighted ensemble (0.70 / 0.30)	M5	Sequence classification ensemble
Scratch BiGRU + RAG (Final)	Bidirectional GRU, attention pooling, RAG context	Final	Val MAP@3 0.996 · Leaderboard ~0.75

2. Introduction

Problem Statement

Given a question prompt and five candidate answers (A–E), the model must rank its top-3 predicted options in order of confidence. The evaluation metric, MAP@3, rewards placing the correct answer as high as possible in the top-3 ranking and gives zero credit if the correct option is not among the top three.

Project Objectives

- **Progressive complexity:** move from classical lexical baselines to transformer-based semantic models, then to retrieval-augmented and fine-tuned architectures.
- **Handle data leakage:** detect and correct for duplicate/near-duplicate questions during train/validation splitting.
- **External knowledge injection:** use RAG (FAISS-indexed Wikipedia passages) to compensate for a small, from-scratch model with no pretrained weights.
- **Reproducibility & tracking:** full seed control, W&B experiment tracking, and checkpoint-based model selection.
- **Deliver a competitive submission:** generate a ranked top-3 prediction for every test-set question in the required Kaggle submission format.

3. EDA of Dataset & Preprocessing (dedupe split)

The raw training data (train.csv) contains 2,000 labeled MCQ rows with columns prompt, A, B, C, D, E, and answer. The hidden test set (test.csv) contains 500 unlabeled rows in the same format.

Exploratory Data Analysis — Key Findings

Metric	Value
Total rows	2,000
Null values	0
Duplicate prompts (exact match)	242 rows / 212 unique prompts affected
Unique option texts	1,584 / 10,000
Correct option — average length	181.9 characters
Wrong option — average length (mean)	160.7 characters
Longest option is correct	40.6% of questions
Top-3 by length captures correct answer	50% of the time (length-bias signal)
Average tokens per question	236 (max 651)

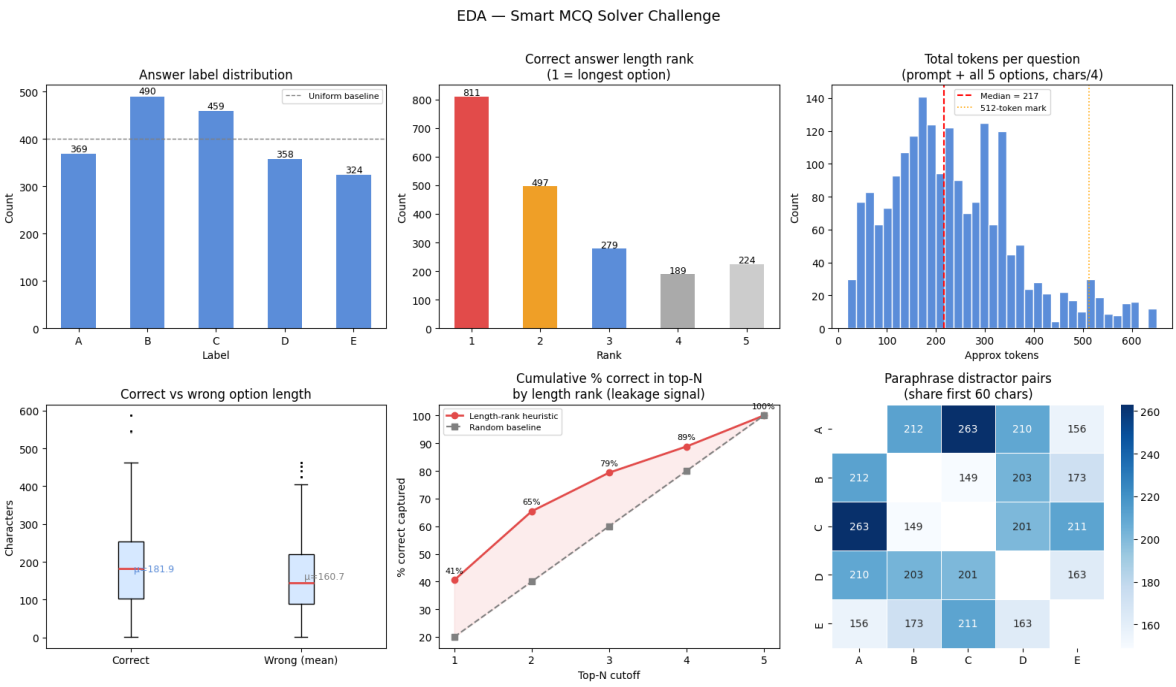


Figure 1 — EDA overview: answer distribution, length-rank bias, token counts, and paraphrase-distractor heatmap.

Duplicate-Prompt Leakage & Prompt-Level Split

The same underlying question can appear multiple times in the dataset, with its five options reshuffled into different column positions (and occasionally reworded) each time. A naive row-level random split can place two copies of the same question — one in train, one in validation — allowing the model to memorize prompt-to-answer mappings rather than learning to reason, which inflates validation MAP@3 without reflecting true generalization.

To prevent this, the dataset was grouped by exact prompt text first, and the train/validation split (75% / 25%, stratified by answer label) was performed on the 1,758 unique prompts rather than on individual rows. Every duplicate row of a given prompt was then assigned to the same split, verified by checking prompt-set intersection.

Split step	Result
Unique prompts	1,758 (of 2,000 total rows)
Prompts appearing more than once	212
Train rows (post-split)	1,503
Validation rows (post-split)	497
Prompt overlap between train & val	0 (verified)

4. Modeling & Experimentation (as per Milestones)

Milestone 1 — NLP foundation and semantic similarity between prompts and options

- Text cleaning (lowercasing, whitespace normalization) while deliberately preserving paraphrase prefixes and scientific symbols to avoid train/test mismatch.
- Two tokenization strategies: TF-IDF (joined string, stopwords removed) and Word2Vec (token list, stopwords kept, 100-dim vectors via mean pooling).
- Cosine similarity between the prompt vector and each of the 5 option vectors, ranked to produce a top-3 prediction.
- MAP@3 established as the core evaluation metric for every subsequent milestone.
- Public leaderboard score for this approach: 0.30 — well below the 0.50 captured by the simple EDA length-rank heuristic (Section 3), showing that lexical/co-occurrence similarity alone fails to capture answer correctness. This motivated the move to pretrained transformer-based approaches in Milestone 2.

Milestone 2 — Pretrained Transformer Approaches

Three pretrained approaches are compared without any task-specific fine-tuning:

- BERT / RoBERTa — [CLS]-token pooling for prompt and option embeddings.
- Sentence-Transformers (SBERT, e.g. all-MiniLM-L6-v2) — direct fixed-size sentence embeddings with cosine similarity scoring.
- Zero-shot classification — a BART-MNLI pipeline scores each option as an entailment hypothesis against the prompt, with no training required. Public leaderboard – 0.49 for zeroshot but yet it is less than length heurisitic approach in EDA.

Milestone 3 — Context Augmentation with RAG

Introduces external knowledge retrieval to give the model additional grounding beyond the prompt and options alone:

- A FAISS similarity index built over Wikipedia passages, encoded with a sentence-embedding retriever model.
- For each prompt, the top-k nearest Wikipedia passages are retrieved and concatenated into a rag_context field.
- A Flan-T5 reader model is used to generate/score answers conditioned on the retrieved context, prompt, and options. Public leaderboard of 0.50+

Milestone 4 — Multiple-Choice Classification & LoRA Fine-Tuning

Moves from embedding-similarity scoring to genuine supervised fine-tuning:

- BERT is adapted to the HuggingFace multiple-choice classification head (one score per option, softmax over 5 options).
- LoRA (Low-Rank Adaptation) is used for parameter-efficient fine-tuning — only small rank-decomposition matrices are trained, keeping the base BERT weights frozen.
- Split performed on unique prompts (same leakage-prevention logic as the final model) before fine-tuning. Public leaderboard of (0.50-0.73) but less than required cutoff.

Milestone 5 — Sequence Classification & Model Ensembling

Two larger pretrained encoders are independently fine-tuned as 5-way sequence classifiers and then combined:

- DeBERTa and RoBERTa are each fine-tuned end-to-end for direct option classification.
- The two models are combined via a weighted ensemble (DeBERTa 0.70 / RoBERTa 0.30), with the weighting tuned against validation MAP@3. Public leaderboard of 0.73+

5. RAG-Augmented BiDirectional GRU (Scratch)

The production model deliberately avoids any pretrained weights — embeddings, recurrent layers, and classification head are all trained from scratch on the 1,503-row training split — with RAG-retrieved Wikipedia context fed in alongside the prompt and each option to compensate for the small data size.

Vocabulary & Encoding

Component	Configuration
Vocabulary size	5,541 words (min. frequency = 2, built from prompt + rag_context + options)
Special tokens	<pad>, <unk>, <sep>
Sequence layout	[prompt] <sep> [context, max 40 tokens] <sep> [option]
Max sequence length	160 tokens (truncated / zero-padded)
Word dropout (train only)	10% random tokens replaced with <unk>

Model Architecture — OptionScorer

```
OptionScorer(
  (embedding): Embedding(5541, 100, padding_idx=0)
  (embed_drop): Dropout(p=0.4, inplace=False)
  (gru): GRU(100, 64, batch_first=True, bidirectional=True)
  (attn): Linear(in_features=128, out_features=1, bias=True)
  (fc): Sequential(
    (0): Linear(in_features=128, out_features=64, bias=True)
    (1): ReLU()
    (2): Dropout(p=0.4, inplace=False)
    (3): Linear(in_features=64, out_features=1, bias=True)
  )
)
```

Training Strategy

Setting	Value
Optimizer	Adam (lr = 1e-3, weight_decay = 3e-4)
Loss function	CrossEntropyLoss (label_smoothing = 0.1)
LR Scheduler	ReduceLROnPlateau (factor = 0.5, patience = 2, on val_loss)
Gradient clipping	max_norm = 2.0 (prevents exploding gradients in the GRU)
Batch size	32 (train) / 64 (validation & test)
Epochs / early stopping	25 epochs run, patience = 5 on validation MAP@3
Experiment tracking	Weights & Biases (project 24f1002692-t22026)

6. Performance & Comparative Analysis

MAP@3 is used as the primary evaluation metric throughout, consistent with the Kaggle competition's scoring criterion.

Training Performance — Final BiGRU-RAG Model

Epoch	Train Loss	Train MAP@3	Val Loss	Val MAP@3
1	1.608	0.533	1.607	0.526
5	1.472	0.709	1.277	0.679
10	1.037	0.946	0.775	0.924
15	0.850	0.978	0.648	0.974
20	0.738	0.992	0.592	0.985
25 (best)	0.672	0.999	0.534	0.996

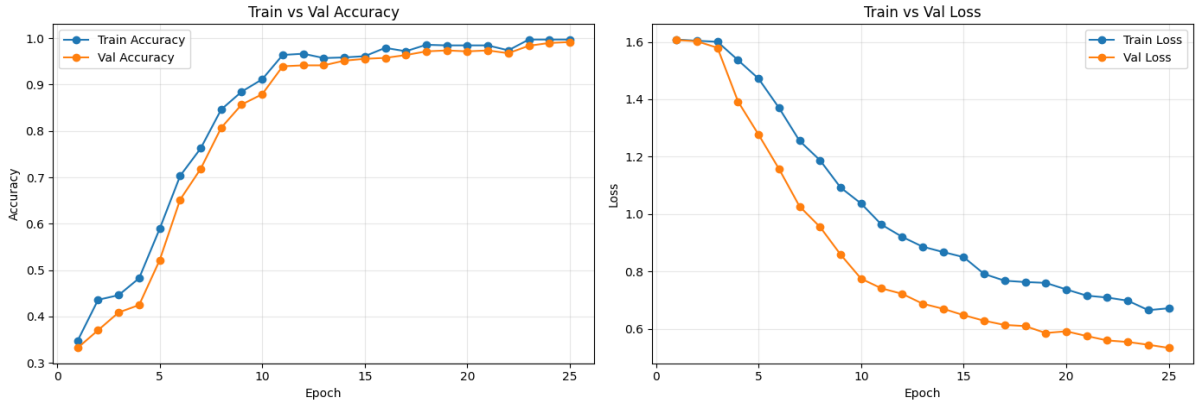


Figure 2 — Train vs. validation accuracy and loss over 25 epochs. Curves track closely with no sign of divergence, indicating the prompt-level split is not leaking within this validation set.

Generalization Gap — Validation vs. Leaderboard

Although the prompt-level split eliminated exact-duplicate leakage, validation MAP@3 (0.996) remained far higher than the confirmed public leaderboard score of approximately 0.75. This gap points to a distribution mismatch between the train/validation pool and the hidden Kaggle test set, rather than residual leakage — the model generalizes well within the training distribution's question style but encounters materially different question phrasing or topic coverage on the actual test set. A submission was accepted once it cleared the competition's 0.73 MAP@3 qualification cutoff.

Metric	Score
Validation MAP@3 (in-notebook)	0.996
Public leaderboard MAP@3	≈ 0.75
Competition qualification cutoff	0.73

7. Conclusion & Future Work

Key Learnings

- Prompt-level (not row-level) splitting is essential once duplicate or near-duplicate questions are present — row-level splits silently inflate validation metrics.
- RAG-retrieved context provides meaningful signal even for a from-scratch model with no pretrained weights, partially compensating for the small (2,000-row) training set.
- Attention pooling with an explicit padding mask is necessary for variable-length sequences batched to a fixed max length — without it, padding tokens dilute the pooled representation.
- Option-order shuffling during training is a simple but effective augmentation against positional shortcut learning.
- A strong validation score does not guarantee a matching leaderboard score — distribution shift between validation and hidden test data must be checked, not assumed away by a clean split.

Challenges Faced

- Diagnosing whether duplicate rows were exact option-letter reshuffles or paraphrased near-duplicates, and confirming the split logic was robust to both.
- RNG non-determinism across runs, resolved with full seed blocks (Python, NumPy, PyTorch, CUDA, cuDNN determinism flags).
- Explaining and closing the gap between near-perfect validation MAP@3 and a materially lower leaderboard score.
- Balancing model capacity (embed_dim, hidden_dim) against the small dataset size to avoid overfitting on a from-scratch architecture.